

COMP1009

Programming Paradigms

Lecture 00-6

Exceptions & PGPFFile

Key elements of OO

- **Classes**
 - Blueprints for objects
 - What data (state) objects of that type contain
 - What methods (functions) can act on objects of that type
- **Objects** – classes specify the format of these
- Set of **methods**/functions which act on the objects
- Set of **data** items in the object (the **state**)
- **Relationships** between objects
 - HAS-A: One object a part of another object?
 - Only exists while container does (Composition)
 - Exists anyway, but conceptually a part (aggregation)
 - USES: Interaction – objects using or creating each other
 - IS-A: Type of – one class is a type of another

Coursework 2b comments

- Coursework 2b will appear on the moodle page today
- Your repos will appear today for you to submit to
 - Repo name has a cw2b_ prefix this time, e.g. cw2b_pszja
- Download the zip of the source code from moodle page
- Add your file that you create, then test your program
- Submit only your Cw2b.java file to the submission system
- Checksubmission pipeline process will check it for you and tell you about most errors you could make
 - Artifacts tell you exactly what I compiled – including my test code
- I have found some problems where students obviously did not check the results of checksubmission in coursework 2a
 - E.g. files in subdirectory, or in a package
- Check the “checksubmission” results this time!
- If checksubmission fails, so will marking, and you will get a 0 as your submission does not compile/run

Today's lecture

- Class Diagrams
- Understanding PGPFFile
 - We need to know what an exception is
 - Exceptions use sub-classing so we needed to know about inheritance

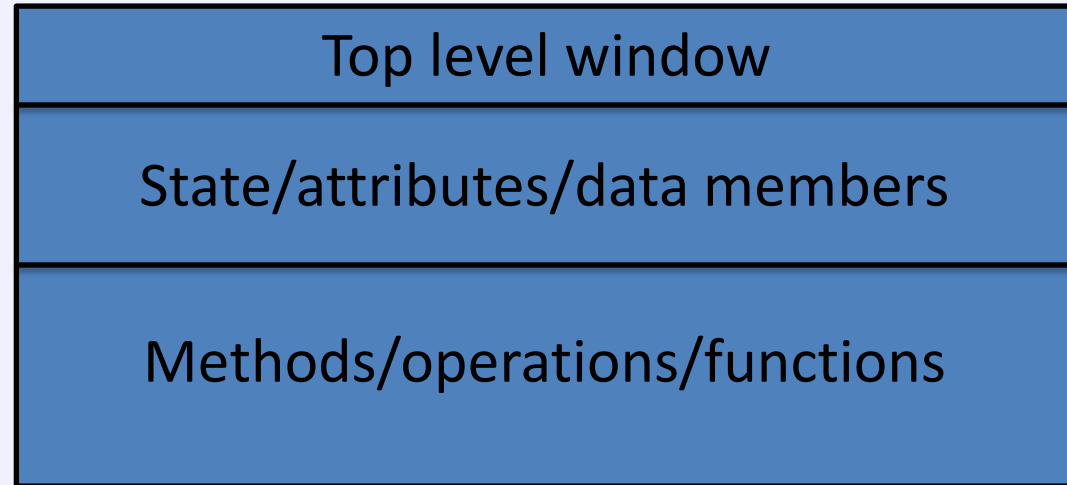
An Introduction to Class Diagrams

What are Class Diagrams?

- We have seen inheritance and aggregation
- Let's look at how we can visualise these relationships – to improve understanding
 - These will be key for understanding design patterns
- We can use diagrams to represent the relationships between classes
 - To help us to understand the structure of the program
 - **Association** is an important relationship – objects of one class *use* objects of another class
 - **Aggregation** and **composition** are also important relationships: one type *has* objects of another type
 - **Inheritance** is the final important relationship that we will think about: one class *is* a specialisation of another

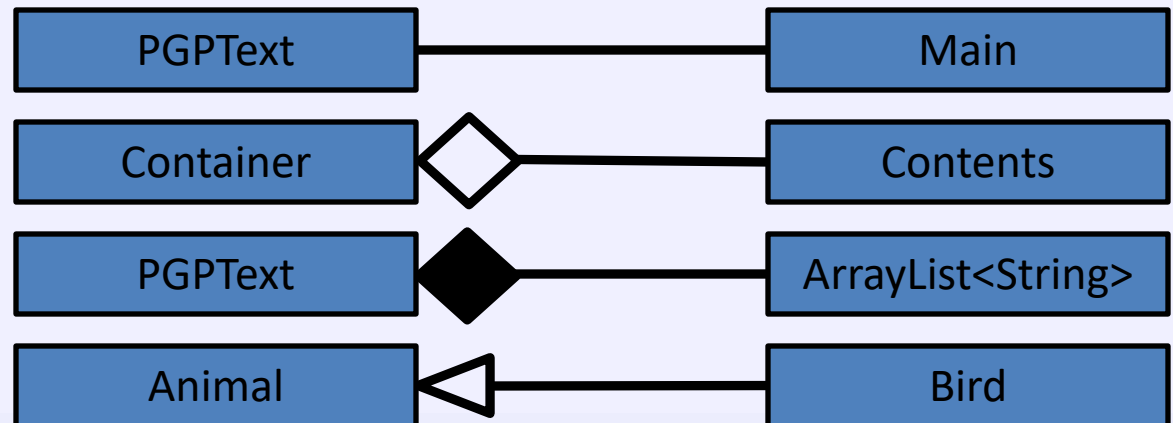
Class diagrams - components

- Box for each class
 - Three sections:
 - Class name
 - Member data
 - Methods/functions



- Line for each relationship – symbol-end matters!

- Association
- Aggregation
- Composition
- Is-A / type of



Reminder: Animal and Bear sub-class

```
public class Animal
{
    public String getType()
    {
        return "Animal";
    }

    public String getName()
    {
        return "I am an animal";
    }

    public String getNoise()
    {
        return "Unknown noise";
    }
}
```

```
public class Bear extends Animal
{
    public String getName()
    {
        return "I am a bear";
    }

    public String getNoise()
    {
        return "GROWL...";
    }
}
```

Use 'extends' to get inheritance,
so a **Bear** "IS-AN" **Animal**
It gets everything in **Animal** and
can change it or add to it

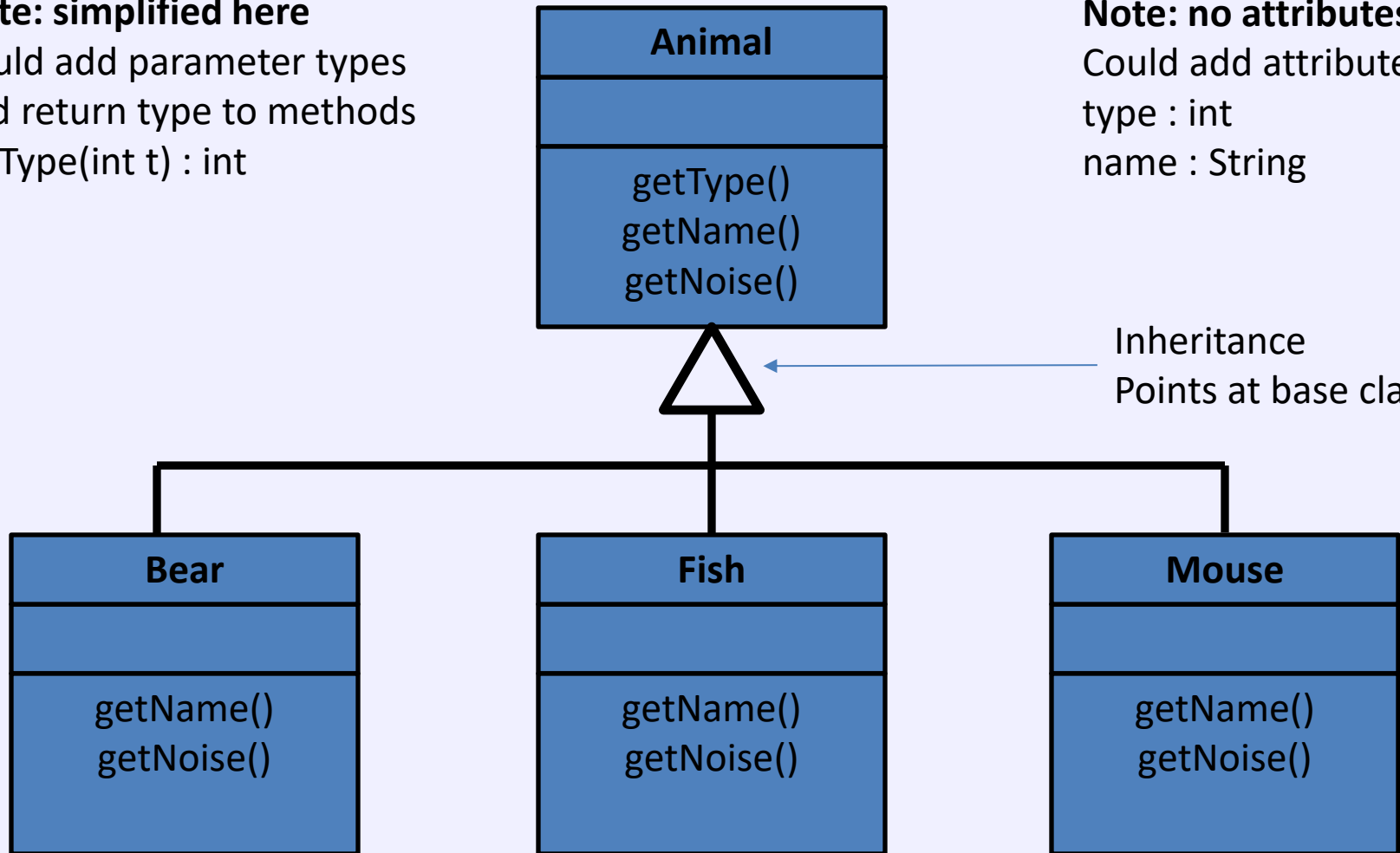
Class diagram for Animals

Note: simplified here

Could add parameter types
and return type to methods
setType(int t) : int

Note: no attributes here

Could add attributes
type : int
name : String



Reminder: the PGPText class

```
import java.util.ArrayList;

public class PGPText
{
    public PGPText()
    {
        internalArray =
            new ArrayList<String>();
    }

    public int getLineCount()
    {
        return internalArray.size();
    }

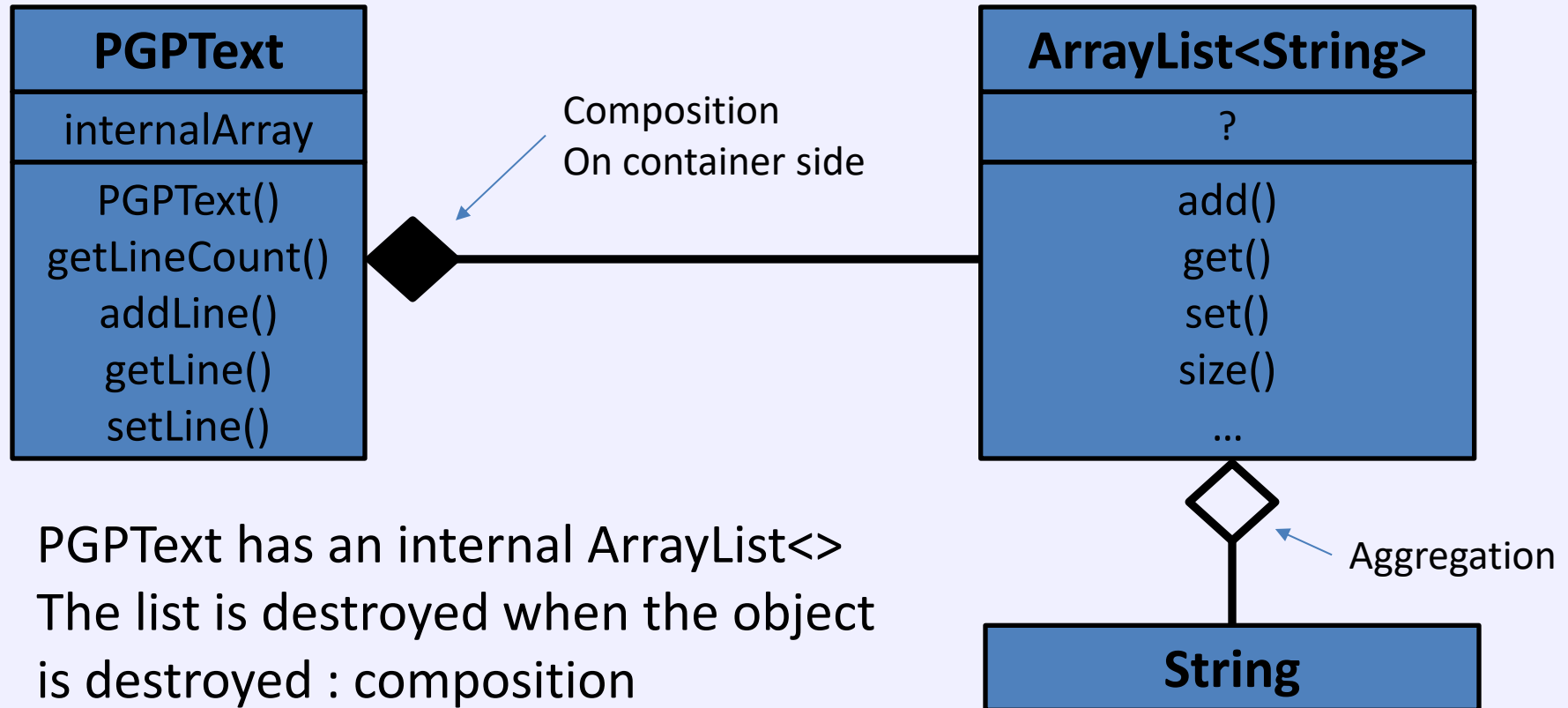
    // A standard collection class
    protected ArrayList<String>
        internalArray;
```

```
    public void addLine( String str )
    { internalArray.add(str); }

    public String getLine( int iLine )
    {
        return internalArray.get(iLine);
    }

    public void setLine(
        int iLine, String strText )
    {
        while ( iLine >=
            internalArray.size() )
            internalArray.add("");
        internalArray.set(iLine, strText);
    }
}
```

Class diagram for PGPText



- PGPText has an internal ArrayList<>
- The list is destroyed when the object is destroyed : composition
- The ArrayList<> contains Strings
- These Strings exist outside of the ArrayList<> but references to them are added to the array: aggregation

Sometimes we omit the attributes and methods
Especially for standard classes

Aside: The 'Object' class

- All classes in Java are subclasses of class 'Object'
 - If you don't say what a class extends then it extends the class called Object
 - All objects are Object (sub-classes) – direct or indirect
- The class 'Object' has a few methods of its own
 - Therefore, every class inherits these and can add their own implementation, E.g.
 - **toString()** : get a **String** object to represent it
 - **equals(Object obOther)** : determine whether two objects are equivalent / the same
 - **hashCode()** : get a **unique** id for the object (e.g. from its address in memory, or its content)
 - **getClass()** : get a 'class object' for the object
 - Aside: can also do: **if (ob instanceof ClassName)**

Exceptions

When functions want to report
exceptional circumstances
And an example of inheritance

Functions call other functions...

```
void fn1()
{
    doSomething1...
    fn2();
    doSomething2...
}
```

```
void fn2()
{
    doSomething3...
    fn3();
    doSomething4...
}
```

```
void fn3()
{
    doSomething5...
    fn4();
    doSomething6...
}
```

```
void fn4()
{
    doSomething7...
    throw new Exception();
    doSomething8...
}
```

Question: Will doSomething2, doSomething4, doSomething6 be done?

What are Exceptions?

- **Exceptions are like extra return value types to report exceptional events**
 - Think of **throwing an exception** as ‘returning’ an Exception object
 - Or a subclass of an exception, since sub-class means ‘IS-A’
- **When an exception is thrown, the function will end unless the exception is caught**
 - **Then the caller function will end unless the exception is caught**
 - This will continue - functions will keep ending immediately until the exception is caught
- This *could* be a problem (unexpected return from a function)
 - If your function COULD raise an exception (i.e. you throw the exception or call function which can) then you **must either**:
 1. **Catch the exception** in your function
 2. **Label the function** so that it is clear that it raises an exception
 - This way at least the caller knows of the risk
 - Note: this is why exceptions are much safer in Java than C++: Sun fixed the issues with C++ exceptions when they wrote Java

Demo code... (for copy-paste)

```

public class MainTestException
{
    void doSomething1() { System.out.println( "\tdoSomething1()"); }
    void doSomething2() { System.out.println( "\tdoSomething2()"); }
    void doSomething3() { System.out.println( "\t\t doSomething3()"); }
    void doSomething4() { System.out.println( "\t\t doSomething4()"); }
    void doSomething5() { System.out.println( "\t\t\t doSomething5()"); }
    void doSomething6() { System.out.println( "\t\t\t doSomething6()"); }
    void doSomething7() { System.out.println( "\t\t\t\t doSomething7()"); }
    void doSomething8() { System.out.println( "\t\t\t\t doSomething8()"); }

    void fn1( int i ) throws Exception
    {
        System.out.println( "fn1("+i+")");
        doSomething1();
        fn2(i);
        doSomething2();
    }

    void fn2( int i ) throws Exception
    {
        System.out.println( "\tfn2("+i+")");
        doSomething3();
        fn3(i);
        doSomething4();
    }

    void fn3( int i ) throws Exception
    {
        System.out.println( "\t\tfn3("+i+")");
        doSomething5();
        fn4(i);
        doSomething6();
    }
}

```

```

void fn4( int i ) throws Exception
{
    System.out.println( "\t\t\tfn4("+i+")");
    doSomething7();
    if ( i > 0 )
        throw new Exception("This is an exception from fn4");
    doSomething8();
}

public void test()
{
    try
    {
        fn1( 0 );
    }
    catch( InterruptedException e )
    {
        System.out.println( "InterruptedException: " + e.getMessage() );
    }
    catch( Exception e )
    {
        System.out.println( "Exception: " + e.getMessage() );
    }
}

// You could put this main function in another class if you wished
public static void main(String[] args)
{
    // Create a new object
    MainTestException ob = new MainTestException();
    // Call test on it
    ob.test();

    // Or do that in one line...
    //new MainTestException().test();
}

```

16

Exceptions summary

- Exceptions are a really powerful feature
 - They allow functions to report error conditions that can be handled, without having to have special 'return values' with specific meanings
- But, they are dangerous to your program
 - Not catching an exception can end your program
- Because of this you need to know:
 - Exceptions exist and what they are for
 - That they are commonly used in Java to report errors
 - How to catch them, so your program will not exit
- I don't ask you to memorise class library details
 - Don't learn a list of exception class names

The code for exceptions...

- Something 'throws' an exception (or any 'Throwable' object)
`throw new InterruptedException("message text");`
- Catch exception (or subclass) – catch clauses considered in order

```
try
```

```
{
```

```
    foo1( i );           // foo could throw an exception
```

```
}
```

```
catch( InterruptedException e )
```

```
{
```

```
    System.out.println( "Interrupted: " + e.getMessage() );
```

```
}
```

```
catch( Exception e )
```

```
{
```

```
    System.out.println( "Exception: " + e.getMessage() );
```

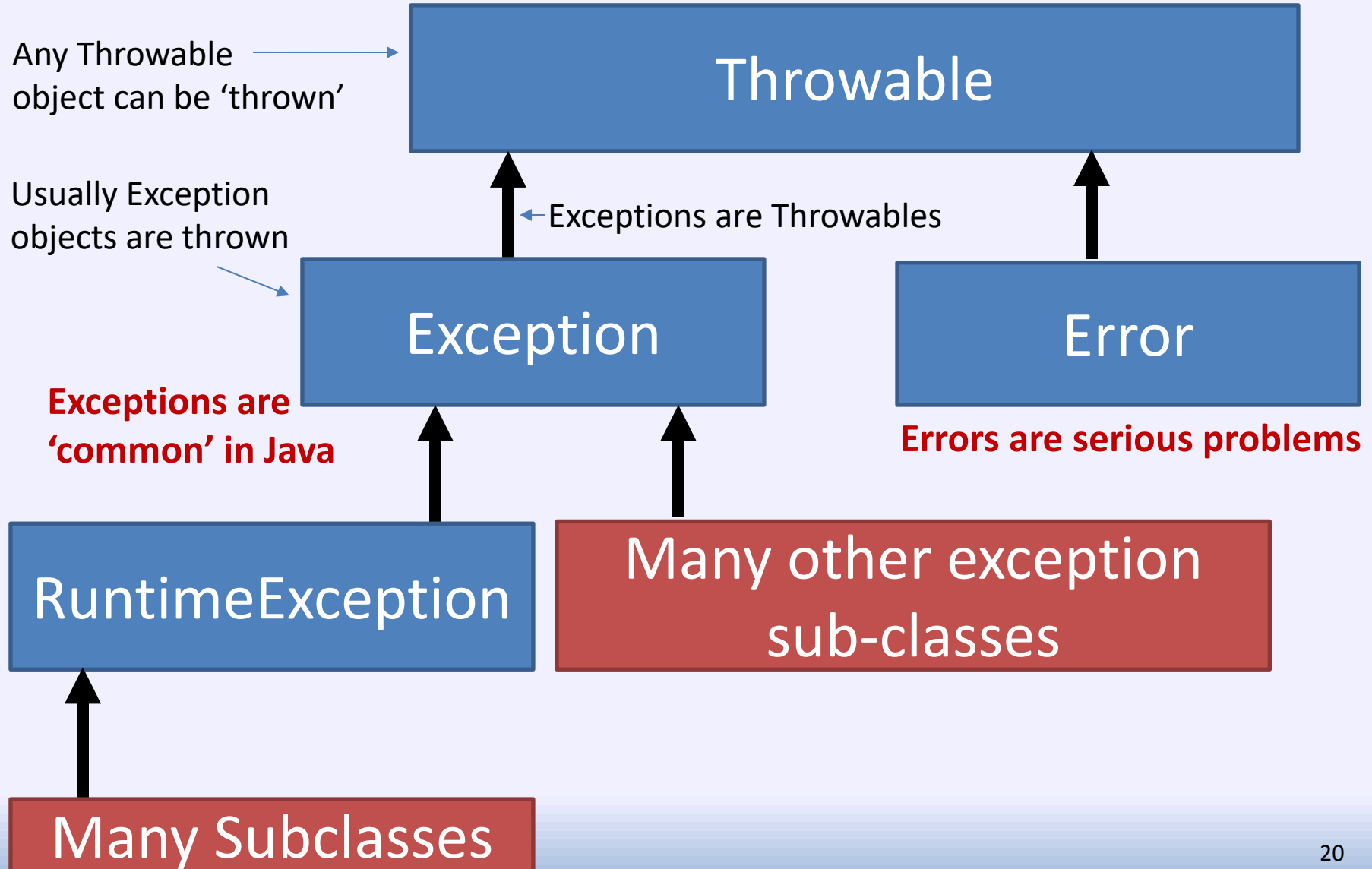
```
}
```

- **You MUST label the function if you don't catch the exceptions**
`public static void foo1( int i ) throws InterruptedException`

Background info: Throwable

- You can throw any **Throwable** object, not just an Exception object
- **Throwable** has two sub-classes
 - **Exception**
 - **Error**
- **Error** indicates a serious problem “that a reasonable application should not try to catch”
 - These indicate “abnormal conditions that should never occur”
 - See for more information:
<https://docs.oracle.com/javase/8/docs/api/java/lang/Error.html>
- So, in general, please throw **Exception** subclass objects if you need to throw an exception

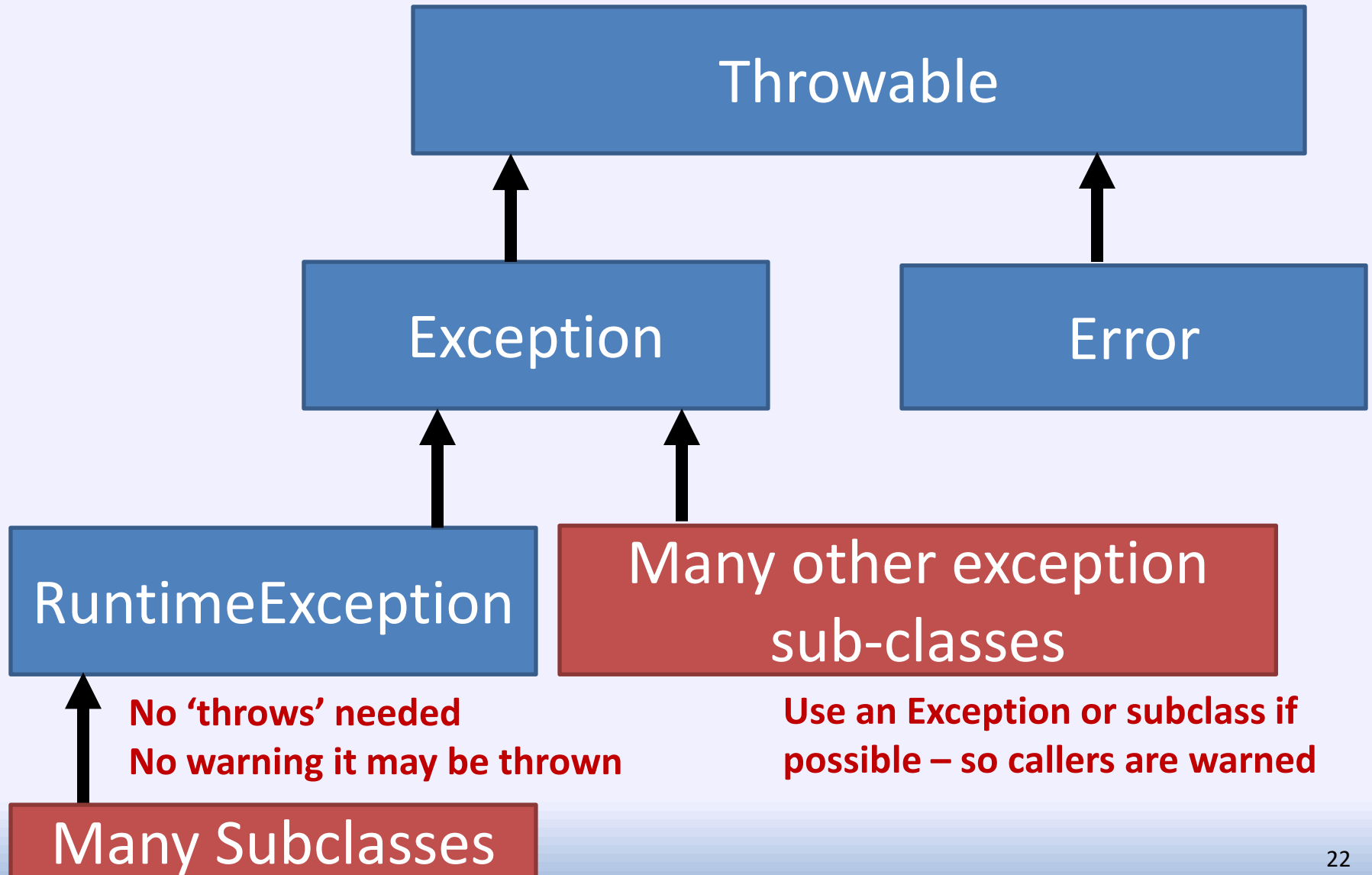
Key Standard Throwable Classes



Special case: RuntimeException

- **RuntimeException** is a special subclass of Exception
 - Covers things that are not predictable or are common from many places and you don't want to have to always check for E.g.
 - **IndexOutOfBoundsException**
 - **IllegalArgumentException**
- You **do not** have to catch or declare that you are throwing a **RuntimeException** (or subclass) that you throw
 - This is why you can call functions which can throw **IndexOutOfBoundsExceptions**, for example, without you having to add a catch, or declare that you may throw it
- In general, avoid throwing a **RuntimeException** subclass object unless you don't want to force the caller to check for the exception – they won't get warned!

Repeat: Key Standard Throwable Classes



Some Standard Java Exception classes

java.lang.[Exception](#) ← You can create your own subclasses of this

java.lang.[CloneNotSupportedException](#)

java.lang.[InterruptedException](#)

java.lang.[ReflectiveOperationException](#)

(I have omitted many of the exception subclasses from this)

java.lang.[RuntimeException](#) ← Note that the following are many subclasses of *RuntimeException*!!!
(Inheritance = IS-A)

java.lang.[ArithmeticException](#)

java.lang.[ArrayStoreException](#)

java.lang.[ClassCastException](#) ← When you cast an object reference

java.lang.[EnumConstantNotPresentException](#)

java.lang.[IllegalArgumentOutOfRangeException](#)

java.lang.[IllegalThreadStateException](#)

java.lang.[NumberFormatException](#)

java.lang.[IllegalMonitorStateException](#)

java.lang.[IllegalStateException](#)

java.lang.[IndexOutOfBoundsException](#) ← Access bad array index

java.lang.[ArrayIndexOutOfBoundsException](#)

java.lang.[StringIndexOutOfBoundsException](#)

java.lang.[NegativeArraySizeException](#)

java.lang.[NullPointerException](#) ← You used a null object reference

java.lang.[SecurityException](#)

java.lang.[TypeNotPresentException](#)

java.lang.[UnsupportedOperationException](#)

Throwing your own exceptions

- You can throw your own exception types to indicate an error that you can't indicate through a return value
- Create (or use an existing) sub-class of **Exception**
 - You can create a new subclass of a relevant exception and throw it
 - Or you can throw an appropriate standard exception type and change the message text appropriately
 - They have constructors allowing you to specify a message
- Particularly important if you want to report an error from a constructor (remember this for JUnit)
 - Constructor has no return value so no other option to return error information to the thing creating the object, will prevent object creation, and give exception to caller

PGPFile class

This class is bigger than PGPText
But apart from the exceptions it
is also a simple class

Abstraction of PGPFile

```
public class PGPFile
{
    // Open a file for reading – fopen( filename, “r”)
    public boolean openReadFile( String strFileName )
    // Read the next line from a file which is open- fgets
    public String readNextLine()
    // Close a file which was open for reading – fclose()
    public boolean closeReadFile()

    // Open a new file for writing to - fopen( filename, “w” )
    public boolean openWriteFile( String strFileName )
    // Write a line to the file - fputs/fprintf(“%s”,...)
    public void writeLine( String strLine )
    // Close a file which is open for writing – fclose()
    public boolean closeWriteFile()
}
```

Data member in PGPFFile

```
public class PGPFFile
{
    public boolean openReadFile(
        String strFileName )
    {
        ...
    }

    protected BufferedReader br;
    Data: hidden inside the class
    Gets initialised to null

    public String readNextLine()
    {
        ...
    }

    public boolean closeReadFile()
    {
        ...
    }
}
```

```
public boolean openWriteFile(
    String strFileName )
{
    ...
}

protected BufferedWriter bw;
Data: hidden inside the class
Gets initialised to null

public boolean writeLine(
    String strLine )
{
    ...
}

public boolean closeWriteFile()
{
    ...
}
```

Implementation: First part of PGPFFile

```
public class PGPFFile
{
    public boolean openReadFile(
        String strFileName )
    {
        if ( br != null )
            closeReadFile();
        try
        {
            br = new BufferedReader(
                new FileReader(strFileName));
        }
        catch(Exception e )
        {
            return false;
        }
        return true;
    }
}
```

Data: hidden inside the class

```
protected BufferedReader br;
```

```
public String readNextLine()
{
    if ( br == null ) return null;
    try
    { return br.readLine(); }
    catch (IOException e)
    { return null; }
}

public boolean closeReadFile()
{
    if (br==null) return false;

    try
    { br.close(); br = null; }
    catch (IOException e)
    { return false; }
    return true;
}
```

A FileReader object is created

The reference to the new FileReader() object is stored

```
try
{
    br = new BufferedReader(
        new FileReader(strFileName));
}
catch(Exception e )
{
    return false;
}
return true;
```

FileReader

?

Equivalent code as two steps:

```
FileReader temp = new FileReader(strFileName);
br = new BufferedReader( temp );
```

A BufferedReader object is created

BufferedReader wraps up the reference from the FileReader
BufferedReader has a constructor which can take a parameter

```
try
```

```
{
```

```
    br = new BufferedReader(  
        new FileReader(strFileName));
```

```
}
```

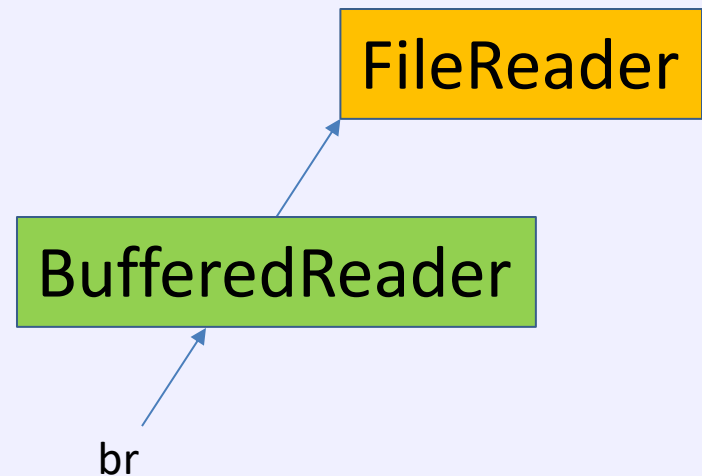
```
catch(Exception e )
```

```
{
```

```
    return false;
```

```
}
```

```
return true;
```



IOException can be thrown – we catch it

java.lang.Object
↑
java.lang.Throwable
↑
java.lang.Exception
↑
java.io.IOException

Not a subclass of RuntimeException

In BufferedReader class it defines the
readLine() method as :

```
public String readLine()  
    throws IOException
```

```
public String readNextLine()  
{  
    if ( br == null ) return null;  
    try  
    { return br.readLine(); }  
    catch (IOException e)  
    { return null; }  
}
```

```
public boolean closeReadFile()  
{  
    if (br==null) return false;  
  
    try  
    { br.close(); br = null; }  
    catch (IOException e)  
    { return false; }  
    return true;  
}
```

IOException can be thrown on closing too

java.lang.Object
↑
java.lang.Throwable
↑
java.lang.Exception
↑
java.io.IOException

Not a subclass of RuntimeException

In BufferedReader class it defines the
readLine() method as :

```
public String readLine()  
    throws IOException
```

```
public String readNextLine()  
{  
    if ( br == null ) return null;  
    try  
    { return br.readLine(); }  
    catch (IOException e)  
    { return null; }  
}
```

```
public boolean closeReadFile()  
{  
    if (br==null) return false;  
  
    try  
    { br.close(); br = null; }  
    catch (IOException e)  
    { return false; }  
    return true;  
}
```

Implementation: second part mirrors first

```
public boolean openWriteFile(
    String strFileName )
{
    if ( bw != null )
        closeWriteFile();

    try
    {
        Same concept, different classes
        bw = new BufferedWriter(
new FileWriter(strFileName));
        return true;
    }
    catch (IOException e)
    {
        return false;
    }
}
```

protected BufferedWriter bw;

Data: hidden inside the class

```
public boolean writeLine(
    String strLine )
{
    if ( bw == null )
        return false;
    try
    {
        bw.write( strLine + "\r\n" );
        return true; }
    catch (IOException e)
    { return false; }
}

public boolean closeWriteFile()
{
    if ( bw == null )
        return false;
    try { bw.close(); bw = null;
        return true; }
    catch (IOException e)
    { return false; }
}
```

Aggregation/composition

- This is another example of aggregation/composition
- Instances of `BufferedReader` and `BufferedWriter` are both used by `PGPFile`
 - Objects are created when the files are opened...

```
br = new BufferedReader(new FileReader(strFileName));  
bw = new BufferedWriter(new FileWriter(strFileName));
```
- The reference to the created object is stored in the relevant member variable
- `FileReader()` and `FileWriter()` are classes to manage the file reading and writing
- `BufferedReader()` and `BufferedWriter()` are classes to buffer the input and output
- Again, we don't care about their implementation, only about how to use them – abstraction again

Reminder: object lifetime in Java

- Java compiler creates .class files from .java files
 - These are platform independent
 - Can compile on one operating system and execute on another
- In Java the JVM (Java Virtual Machine) is responsible for running the .class files
 - Used to be an interpreter
 - Often now a JIT compiler – compiles the code just before it is executed the first time, uses it when re-executed
 - JVM is responsible for allocating the memory needed by new
 - JVM is responsible for freeing the memory when it is no longer needed (it must track what you are using)
 - JVM works out when objects no longer have object references to them – you can't use them anymore anyway
 - This is called Garbage Collection – clearing out the rubbish
 - You can encourage a garbage collection by calling `System.gc();`
 - We set **br = null** when we no longer need the object – the memory/object will be freed at some later time

Next Lecture

- Using a Java GUI class library to create a simple GUI application
- There is an example of using inheritance and aggregation on the next few slides – try it yourself and consider the class diagram to understand what is happening

Exercise: Creating a class using **Composition AND Inheritance**

Try the following code
yourself

Using both composition and inheritance

```
public class PGPTextFile extends PGPText // This is a sub-class of PGPText!
{
    public static void main( String[] args )
    {
        PGPTextFile textfile = new PGPTextFile();
        textfile.load( "input.txt" );
        textfile.save( "output.txt" );
    }

    // We will use this for reading and writing
    // This is a member variable/attribute - we are using composition
    PGPFile file = new PGPFile();

    void load( String filename )
    {
        file.openReadFile(filename);
        String strLine;
        while ( (strLine = file.readLine()) != null )
            this.addLine(strLine);
        file.closeReadFile();
    }

    void save( String filename )
    {
        file.openWriteFile(filename);
        for ( int i = 0 ; i < this.getLineCount() ; i++ )
            file.writeLine( this.getLine(i) );
        file.closeWriteFile();
    }
}
```

Class diagram for PGPTextFile

